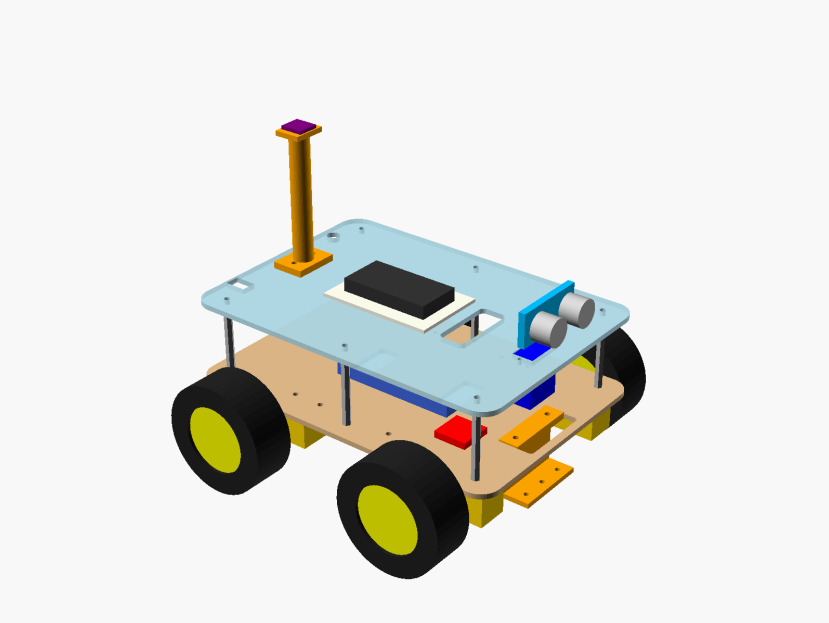
PRIMER ENTREGABLE · PROYECTO FINAL

Robot móvil autónomo basado en ESP32 con brújula digital, detección de colores y de obstáculos, app Streamlit y chatbot J.A.R.V.I.S.

Objetivos específicos – Entrega Corte 2



Ana Sofía Escobar Jaramillo

Noviembre de 2026

Repositorio: github.com/anaescobarj/aco-swarm → robot-esp32/

Contenido

1. Breve introducción **3**

1.1 Alcance de esta entrega . . . 3

2. Objetivos **3**

2.1 Objetivo general . . . 3

2.2 Objetivos específicos (Corte 2) y dónde se cumplen . . . 3

2.3 Meta final del proyecto . . . 4

3. Requerimientos del sistema **4**

4. Arquitectura del robot **5**

4.1 Diagramas de bloques por módulo . . . 5

4.2 Selección y justificación de materiales . . . 6

4.3 Presupuesto de potencia y autonomía . . . 7

5. Diseño mecánico **8**

5.1 Decisiones de diseño . . . 8

6. Diseño electrónico **11**

6.1 Cálculos de diseño . . . 13

7. Construcción de los módulos (firmware ESP32) **13**

7.1 Brújula digital . . . 13

7.2 Detección de colores . . . 14

7.3 Obstáculos y máquina de estados . . . 14

7.4 Comunicación y conexión con el chatbot . . . 15

7.5 Seguridad . . . 15

8. Aplicación Streamlit y chatbot **15**

9. Validación **17**

9.1 Gemelo digital . . . 17

9.2 Pruebas automáticas . . . 18

9.3 Plan de pruebas físicas (siguiente corte) . . . 18

10. Estado y próximos pasos **18**

11. Conclusiones **19**

12. Referencias **19**

1. Breve introducción

Este documento presenta el primer entregable del proyecto final: el diseño de un **sistema robótico móvil basado en ESP32** cuya estructura mecánica y electrónica es desarrollada por la estudiante. El robot opera en dos modos: **manual**, comandado por voz (arriba, abajo, derecha, izquierda) desde una aplicación web Streamlit o con un control remoto propio construido con un segundo ESP32; y **autónomo**, en el que recorre una pista seccionada por colores, se orienta con una brújula digital, detecta en tiempo real los cambios de color de la superficie (negro, blanco, rojo, amarillo, verde y azul) y esquiva obstáculos para luego retomar su trayecto.

La aplicación Streamlit se comunica de forma inalámbrica con el ESP32 (Wi-Fi) y muestra en tiempo real la **dirección polar actual y la deseada**, además de toda la telemetría. Un chatbot estilo **J.A.R.V.I.S.** explica el proyecto e interpreta las variables capturadas en tiempo real.

El informe sigue el orden de los objetivos específicos de la entrega del corte 2: (1) arquitectura, requerimientos, diagramas de bloques y selección de materiales; (2) diseño mecánico (CAD y planos) y electrónico (esquemas); (3) construcción de los módulos electrónicos/firmware; y (4) la aplicación Streamlit con comunicación inalámbrica. Todo el material (firmware, app, simulador, CAD, planos, generadores de figuras y de este informe) está en el repositorio.

1.1 Alcance de esta entrega

Se entrega el diseño completo y el software funcional. Como el hardware aún está en ensamble, la lógica de control se validó con un **gemelo digital** (simulador) que expone exactamente la misma API HTTP del firmware y ejecuta la misma máquina de estados sobre las tres pistas de la meta final. El firmware se verificó sintácticamente; su prueba en la placa física es la primera actividad del siguiente corte (sección 10).

2. Objetivos

2.1 Objetivo general

Diseñar e implementar un sistema robótico móvil basado en ESP32, con estructura mecánica y electrónica desarrolladas por la estudiante, capaz de operar en modos manual (comandos de voz y control desarrollado con ESP32) y autónomo, orientarse mediante una brújula digital, comunicarse de forma inalámbrica con una aplicación móvil/web (Streamlit) que visualiza en tiempo real la dirección polar actual y deseada, detectar en tiempo real cambios de color de la superficie (negro, blanco, rojo, amarillo, verde y azul) y obstáculos (esquivándolos y siguiendo con el trayecto), e incorporar un chatbot tipo J.A.R.V.I.S. que explique el proyecto e indique las variables capturadas en tiempo real.

2.2 Objetivos específicos (Corte 2) y dónde se cumplen

Tabla 1. Trazabilidad entre objetivos específicos y entregables.

#	Objetivo específico	Sección / evidencia
1	Crear la arquitectura del robot, analizando requerimientos funcionales y no funcionales, con diagramas de bloques por módulo y selección/justificación de materiales.	Sec. 3 y 4 · Fig. arquitectura y 9 diagramas de bloques · Tablas de requerimientos y materiales
2	Diseñar la estructura mecánica y el sistema electrónico (modelos CAD, planos mecánicos y esquemas eléctricos).	Sec. 5 y 6 · hardware/cad/chasis.scad, DXF, STL · planos acotados · esquemas
3	Construir los módulos electrónicos que capturan datos, manejan la brújula, detectan obstáculos y colores y se conectan con el chatbot en tiempo real.	Sec. 7 · firmware/robot_esp32 (módulos .h) · API /api/state · validación Sec. 9
4	Desarrollar una app Streamlit inalámbrica (Wi-Fi) para control manual y visualización en tiempo real de la dirección polar actual y deseada.	Sec. 8 · app/app.py · capturas del dashboard, control por voz y J.A.R.V.I.S.

2.3 Meta final del proyecto

Tabla 2. Niveles de dificultad de la meta final (modelados en el simulador).

Nivel	Obstáculos mínimos	Colores de la pista	Pista
1	4	3	Curva cerrada tipo «frijol»
2	6	4	Estrella de 5 puntas
3	8	6	Pista irregular con curvas encadenadas

3. Requerimientos del sistema

Los requerimientos se derivan del objetivo general y de la meta final (tres niveles de pista). Cada uno tiene un criterio de verificación que se reutiliza en el plan de pruebas.

Tabla 3. Requerimientos funcionales.

ID	Requerimiento funcional	Verificación
RF-01	Moverse adelante, atrás, girar a derecha e izquierda con velocidad regulable.	Prueba manual
RF-02	Recibir comandos de voz en español (arriba, abajo, derecha, izquierda, detente, modo, rumbo).	Pruebas unitarias del intérprete (15 frases)
RF-03	Recibir comandos de un control remoto propio basado en ESP32 (ESP-NOW).	Prueba de enlace
RF-04	Medir el rumbo magnético (0–360°) con brújula digital calibrable.	Comparación con brújula
RF-05	Mantener o alcanzar una dirección polar deseada enviada desde la app.	Error < 8°
RF-06	Seguir el trayecto de la pista de forma autónoma.	Simulación niveles 1–3
RF-07	Detectar 6 colores (negro, blanco, rojo, amarillo, verde, azul) y los cambios de sección.	Matriz de confusión en pista
RF-08	Detectar obstáculos frontales (< 30 cm), esquivarlos y retomar el trayecto en el sentido original.	Simulación: 0 colisiones
RF-09	Publicar telemetría en tiempo real (rumbo actual/deseado, distancia, color, línea, motores, batería, estado, eventos).	/api/state
RF-10	Visualizar en la app la dirección polar actual y deseada en tiempo real.	Dashboard
RF-11	Chatbot J.A.R.V.I.S. que explique el proyecto y las variables en tiempo real.	Pestaña chat
RF-12	Cambiar entre modo manual y autónomo; pausa y parada de emergencia.	App / control
RF-13	Calibrar la brújula y las referencias de color desde la app.	Pestaña calibración

Tabla 4. Requerimientos no funcionales.

ID	Requerimiento no funcional	Criterio
RNF-01	Latencia de control	Lazo de control a 20 Hz; comando app→motor < 150 ms
RNF-02	Refresco de la visualización	2–4 Hz en el dashboard
RNF-03	Seguridad	Detención automática si no llegan comandos (600 ms app / 300 ms ESP-NOW); bloqueo de avance a < 12 cm en manual; fusible 3 A; BMS
RNF-04	Autonomía energética	≥ 45 min de operación continua (estimado 1.5 h)
RNF-05	Alcance inalámbrico	≥ 15 m en interiores (Wi-Fi y ESP-NOW)
RNF-06	Dimensiones y peso	≤ 22 × 20 × 18 cm; ≤ 1 kg
RNF-07	Mantenibilidad	Firmware modular (un archivo por módulo), parámetros en config.h, piezas paramétricas en CAD
RNF-08	Usabilidad	Interfaz en español; comandos por voz; funciona en celular y PC
RNF-09	Costo	≤ 450.000 COP en componentes
RNF-10	Robustez del magnetómetro	Montaje a > 10 cm de motores y batería; calibración en NVS

4. Arquitectura del robot

La arquitectura es **centralizada en un ESP32** que ejecuta un lazo de control a 20 Hz (percepción → decisión → actuación) y, en paralelo, atiende la red: un servidor HTTP con API REST/JSON para la app y un receptor ESP-NOW para el control remoto. Los sensores se agrupan por bus: I2C (brújula y color), digital (línea y ultrasonido) y analógico (batería). La potencia se separa en dos dominios — motores a 7.4 V y lógica a 5 V/3.3 V — para que el ruido de los motores no reinicie el microcontrolador.

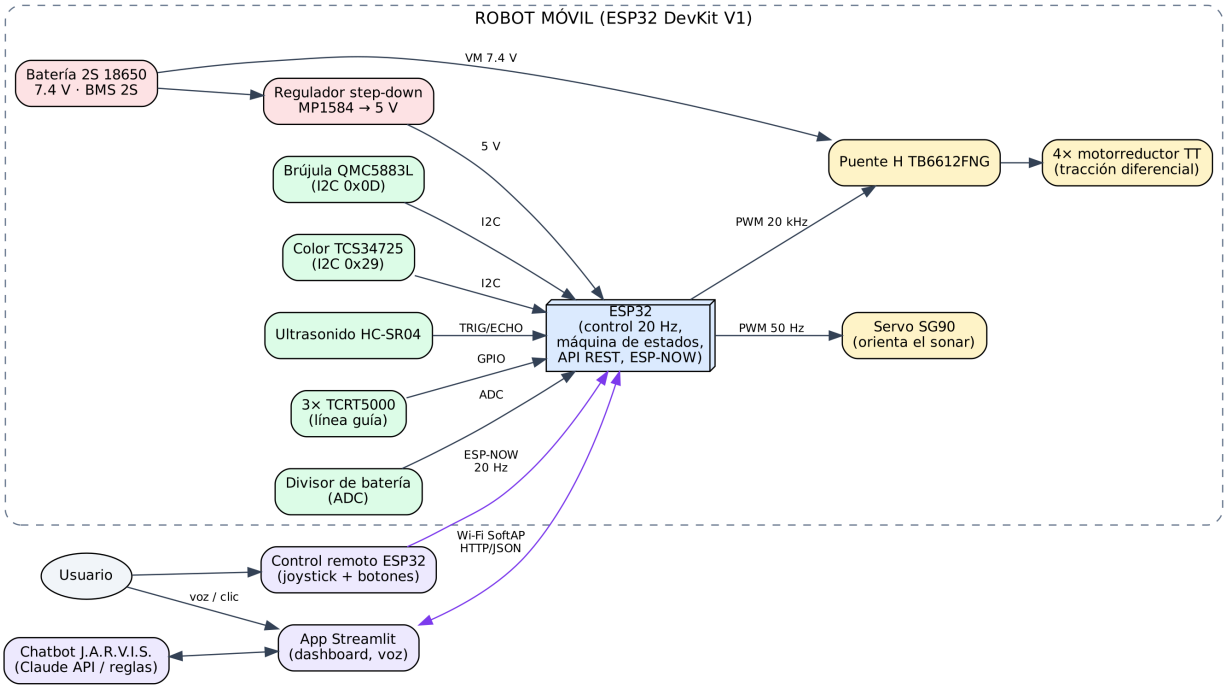


Figura 1. Arquitectura general del sistema.

4.1 Diagramas de bloques por módulo

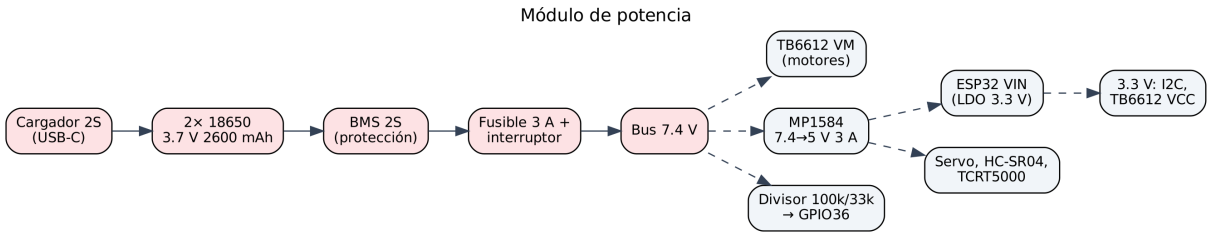


Figura 2. Módulo de potencia: batería 2S protegida, fusible, bus de 7.4 V para motores y regulación a 5 V / 3.3 V para la lógica.

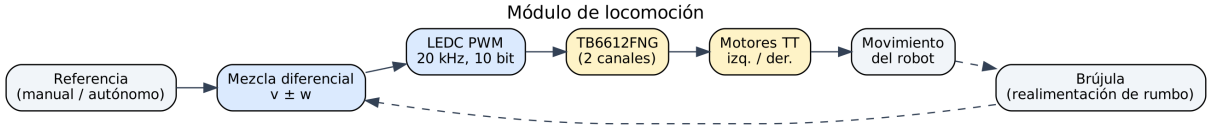


Figura 3. Módulo de locomoción: mezcla diferencial, PWM de 20 kHz y realimentación de rumbo con la brújula.

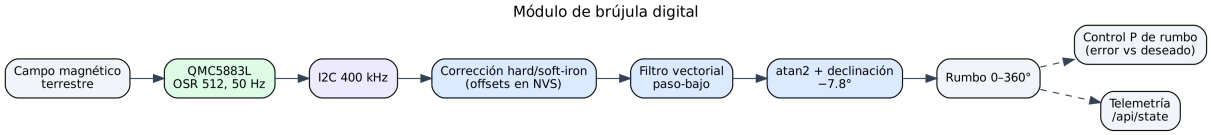


Figura 4. Módulo de brújula digital: lectura, calibración hard/soft-iron, filtrado y cálculo del rumbo.

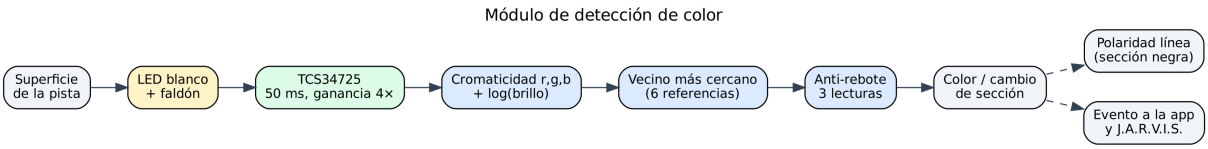


Figura 5. Módulo de detección de color de la superficie.

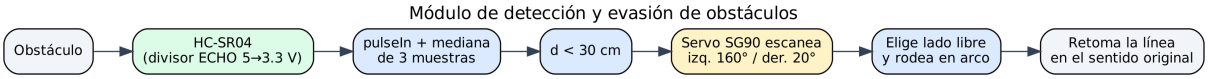


Figura 6. Módulo de detección y evasión de obstáculos.

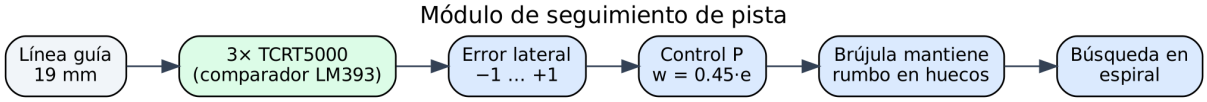


Figura 7. Módulo de seguimiento de la pista.

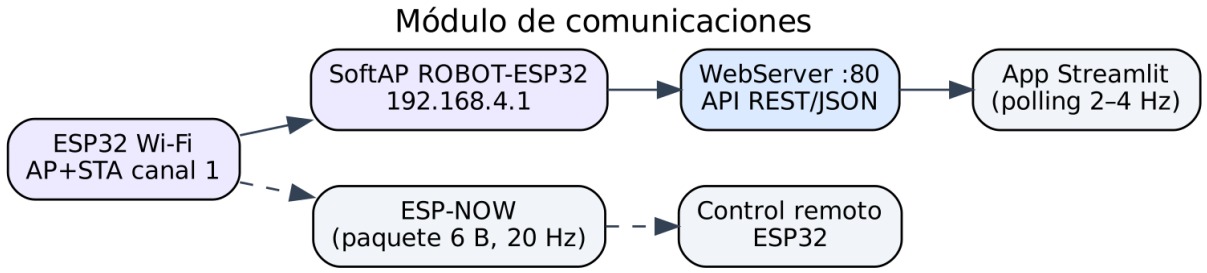


Figura 8. Módulo de comunicaciones inalámbricas (Wi-Fi SoftAP + ESP-NOW).

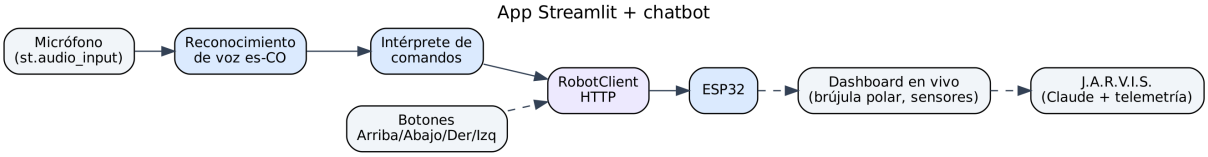


Figura 9. Aplicación Streamlit: voz, control y visualización; chatbot.

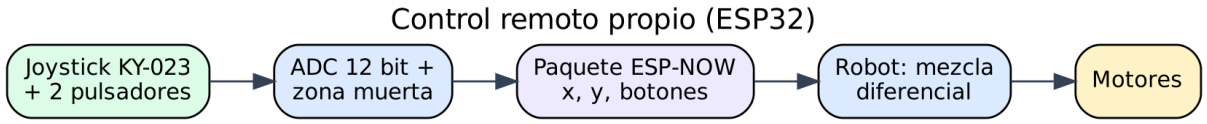


Figura 10. Control remoto propio basado en ESP32.

4.2 Selección y justificación de materiales

Tabla 5. Lista de materiales con alternativas y justificación.

Módulo	Componente (cant.)	Alternativas evaluadas	Justificación de la selección	COP aprox.
Control	ESP32 DevKit V1 (1)	Arduino UNO + HC-05; ESP8266	Wi-Fi y Bluetooth integrados, ESP-NOW, 2 núcleos a 240 MHz, PWM por hardware (LEDC) en cualquier pin, 2 buses I2C. Exigido por el curso y elimina módulos de radio externos.	28.000
Locomoción	Puente H TB6612FNG (1)	L298N; DRV8833	MOSFET: caída de ~0.2 V frente a ~2 V del L298N (más torque y menos calor), 1.2 A continuos / 3.2 A pico por canal, lógica de 3.3 V, PWM hasta 100 kHz. Más pequeño.	15.000
Locomoción	Motorreductor TT 1:48 + rueda 65 mm (4)	N20 con encoder; motores 25GA	Bajo costo, torque suficiente para ~0.9 kg, montaje simple; 4WD da tracción en curvas de la pista. Velocidad ~0.5 m/s a 6 V.	40.000
Brújula	GY-271 QMC5883L (1)	HMC5883L (descontinuado); MPU-9250; BNO055	Magnetómetro 3 ejes de 16 bits, ±8 G, I2C, 3.3 V, barato y disponible. El BNO055 fusiona sensores pero cuesta 4×; con calibración y montaje en mástil el QMC5883L logra ±2–3°.	15.000
Color	TCS34725 con LED (1)	TCS3200 (frecuencia); LDR + LED RGB	Salida digital I2C de 16 bits por canal R,G,B y Clear, filtro IR integrado y LED propio: lecturas estables bajo distinta iluminación. El TCS3200 requiere medir frecuencias y es más sensible a la luz ambiente.	30.000
Obstáculos	HC-SR04 (1) + servo SG90 (1)	VL53L0X (ToF); IR Sharp GP2Y0A21	Rango 2–400 cm, haz de ~30° adecuado para conos; independiente del color del obstáculo. El servo permite escanear izquierda/derecha con un solo sensor para elegir el lado libre.	17.000
Pista	TCRT5000 con comparador (3)	QTR-8RC; un solo sensor	Tres sensores dan error lateral (−1…+1) para un control proporcional; salida digital ajustable por potenciómetro.	12.000
Potencia	2× 18650 2600 mAh + BMS 2S + portapilas	4× AA; LiPo 2S	7.4 V nominales sin regulador para motores, alta capacidad, recargables y con protección (BMS). Más seguras y fáciles de conseguir que LiPo.	45.000
Potencia	Regulador MP1584 (1), fusible 3 A, interruptor, C 470 µF	LM2596; 7805	Conmutado (~92 % eficiencia) y 3 A: soporta picos de servo + Wi-Fi; el 7805 disiparía ~3 W.	12.000
Estructura	MDF/acrílico 3 mm corte láser; piezas PLA; separadores M3	Chasis comercial	Diseño propio (requisito del curso); corte láser preciso y económico; PLA para soportes de sensores a la altura exacta y mástil no magnético.	50.000
Control remoto	ESP32 + joystick KY-023 + 2 pulsadores + 18650 + TP4056	Control RF 2.4 GHz	Reutiliza el ESP32: ESP-NOW tiene latencia < 10 ms y no requiere router.	50.000
Varios	Resistencias, PCB perforada, cables, borneras	—	Divisores de tensión y montaje.	20.000
	Total estimado		Precios de referencia Colombia 2026 (pueden variar).	**334.000**

4.3 Presupuesto de potencia y autonomía

Tabla 6. Consumo estimado por carga.

Carga	Tensión	Corriente típica	Pico
4 motores TT (2 por canal)	7.4 V (PWM ≤ 80 %)	0.8–1.2 A	≈ 3 A (arranque)
ESP32 con Wi-Fi activo	5 V → 3.3 V	120 mA	350 mA (TX)
Servo SG90	5 V	10 mA (reposo)	650 mA (bloqueo)
HC-SR04 / TCRT5000 ×3 / TCS34725+LED / QMC5883L	5 V / 3.3 V	15 / 60 / 22 / 1 mA	—
Total desde la batería (≈)	7.4 V	≈ 1.3 A	≈ 4 A (fusible 3 A lento)

Autonomía estimada: 2.6 Ah × 0.8 (profundidad de descarga útil) / 1.3 A ≈ **1.6 h**, por encima del requerimiento RNF-04. El regulador MP1584 (3 A) cubre con margen el pico de la lógica (≈ 1.1 A), y el TB6612FNG soporta 1.2 A continuos por canal (≈ 0.5 A por canal en operación).

5. Diseño mecánico

El chasis es de **diseño propio**, modelado paramétricamente en OpenSCAD (hardware/cad/chasis.scad): todas las cotas son variables, de modo que se regeneran los planos DXF de corte láser y los STL de impresión 3D con un comando. Tiene dos niveles de 200 × 140 mm unidos por 6 separadores M3 de 45 mm: el inferior (MDF) aloja motores, batería, puente H y regulador (centro de gravedad bajo); el superior (acrílico) aloja el ESP32, el servo con el ultrasonido y el mástil de la brújula.

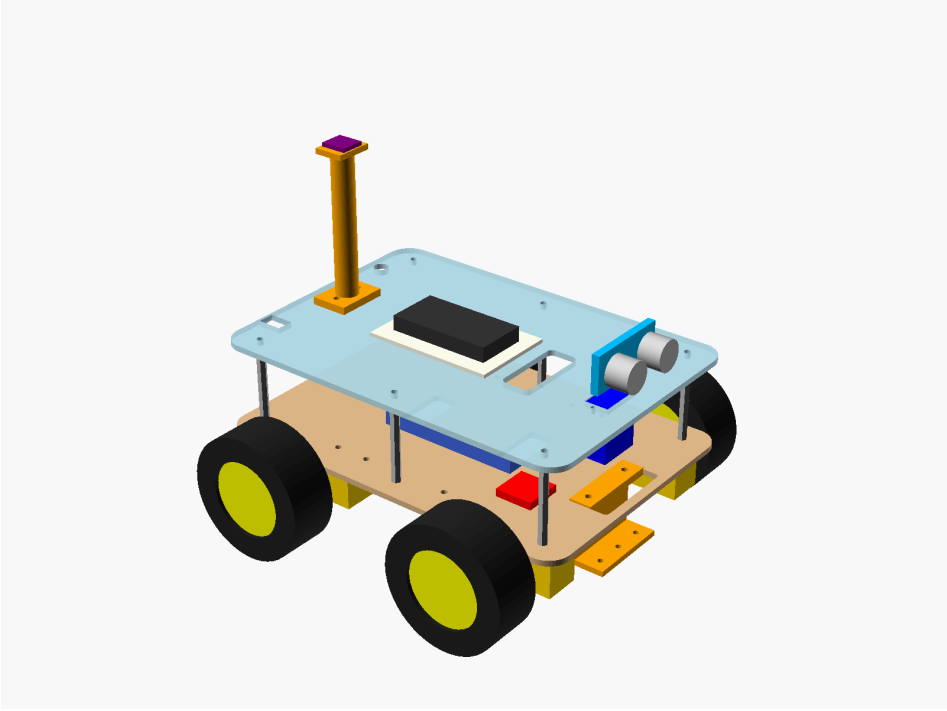


Figura 11. Modelo CAD del ensamble (vista isométrica).

5.1 Decisiones de diseño

- **Mástil de brújula de 80 mm en PLA**, en la parte trasera: el magnetómetro queda a ~125 mm de los imanes de los motores y lejos de la batería, reduciendo la distorsión del campo (sin tornillería metálica cerca).
- **Sensor de color a 5 mm del piso** con faldón que bloquea la luz ambiente y LED propio; **TCRT5000 a 8 mm**, distancia óptima del fotodiodo. Ambos en un soporte impreso atornillado a la ventana frontal de la placa base.

- **HC-SR04 sobre servo SG90** a 110 mm de altura, por encima de las ruedas, para que el haz no detecte el piso y pueda girar ±70° al escanear.
- **Distancia entre ejes 120 mm y vía de 172 mm**: base ancha estable con giro en el sitio (tracción diferencial / skid-steer).
- **Ventanas pasacables** en ambas placas y agujeros de fijación para cada módulo.

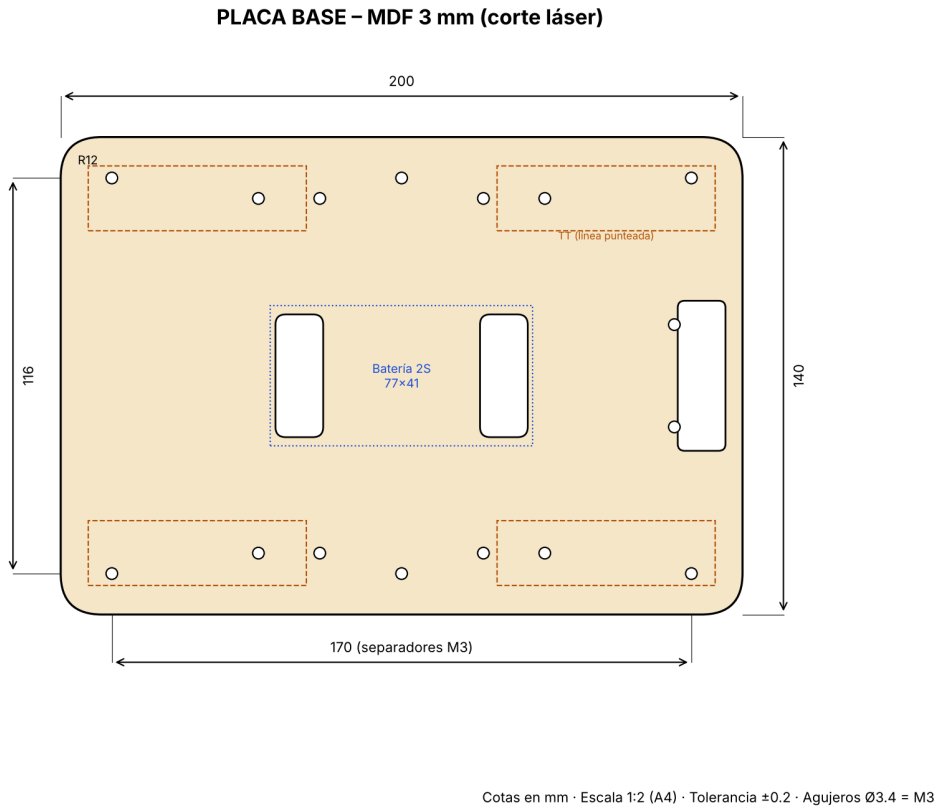


Figura 12. Plano acotado de la placa base (corte láser).

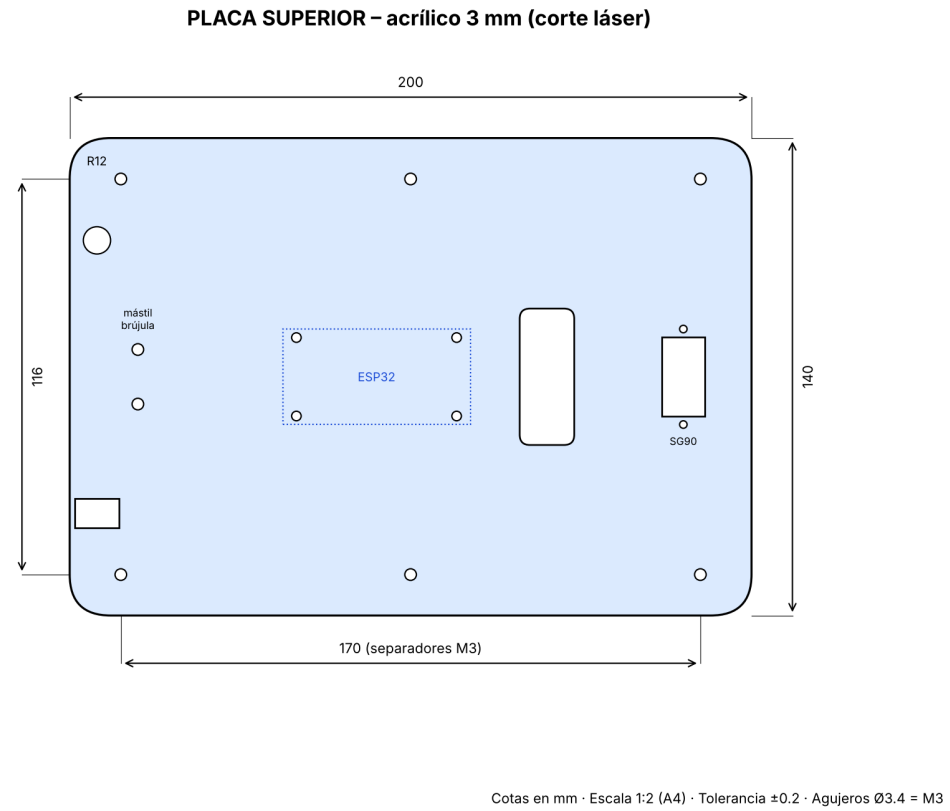


Figura 13. Plano acotado de la placa superior (corte láser).

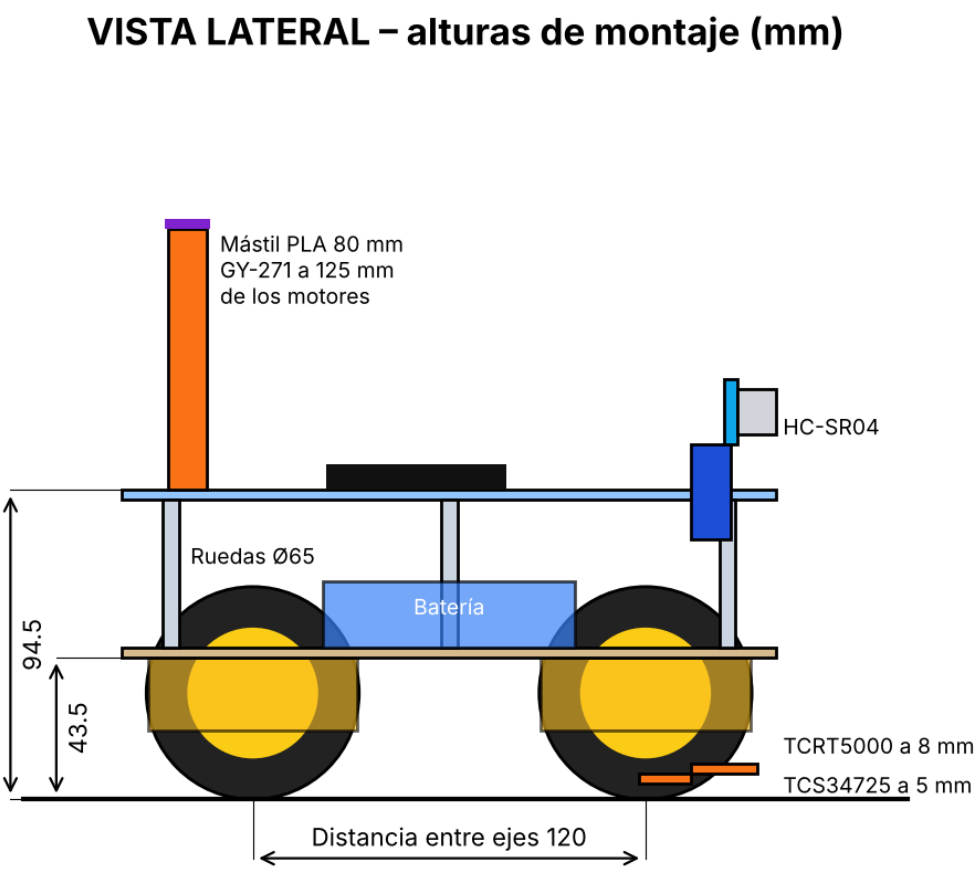


Figura 14. Vista lateral con alturas de montaje de los sensores.

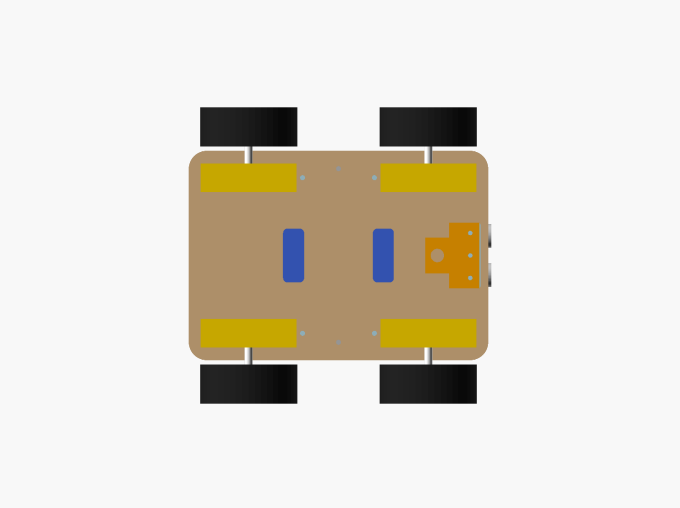


Figura 15. Vista inferior del CAD: motores TT y soporte de sensores de piso.

Tabla 7. Piezas fabricadas.

Pieza	Material / proceso	Archivo
Placa base	MDF 3 mm, corte láser	hardware/cad/placa_base.dxf
Placa superior	Acrílico 3 mm, corte láser	hardware/cad/placa_superior.dxf
Soporte de sensores de piso	PLA, impresión 3D (0.2 mm, 30 %)	hardware/cad/soporte_sensores.stl
Mástil de la brújula	PLA, impresión 3D	hardware/cad/mastil.stl
Escuadra del ultrasonido	PLA, impresión 3D	hardware/cad/soporte_sonar.stl

6. Diseño electrónico

El circuito se divide en la etapa de potencia (Figura siguiente) y las conexiones de señal entre el ESP32 y cada módulo, representadas con etiquetas de red (nets): pines con el mismo nombre están conectados. Se evitaron los pines de arranque (GPIO0, 12, 15) y se usaron los pines solo-entrada 34–39 para sensores.

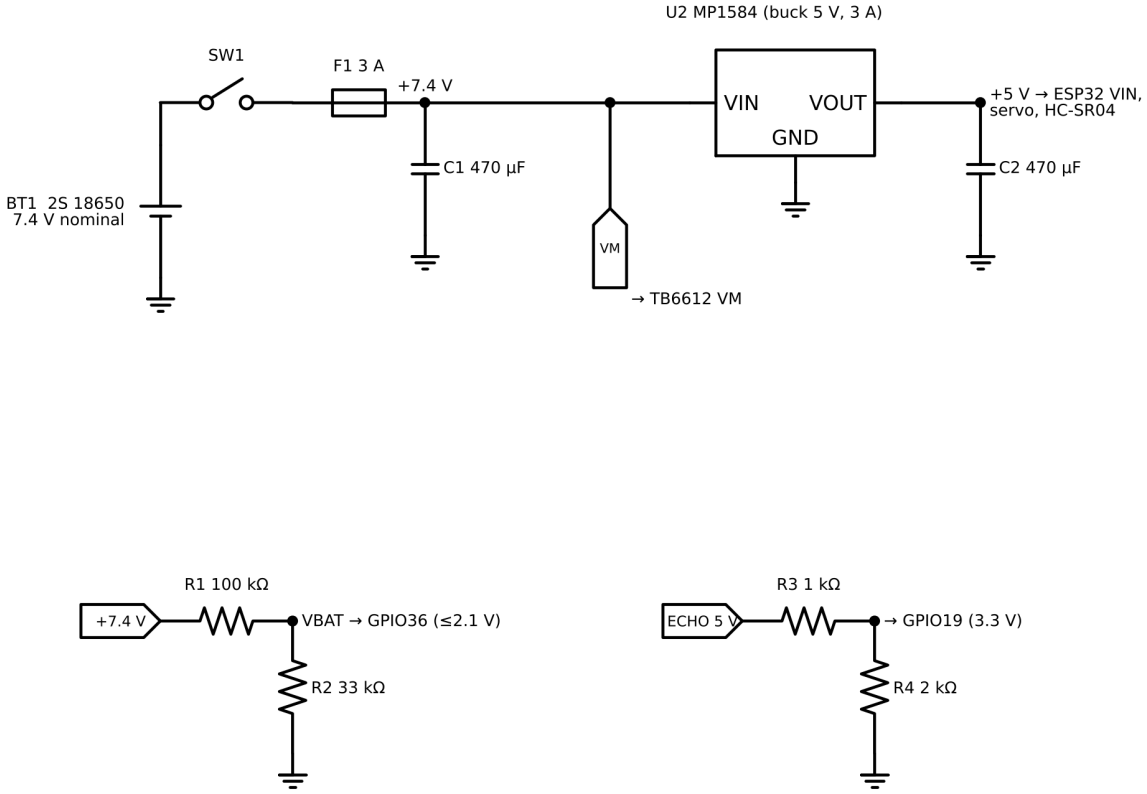
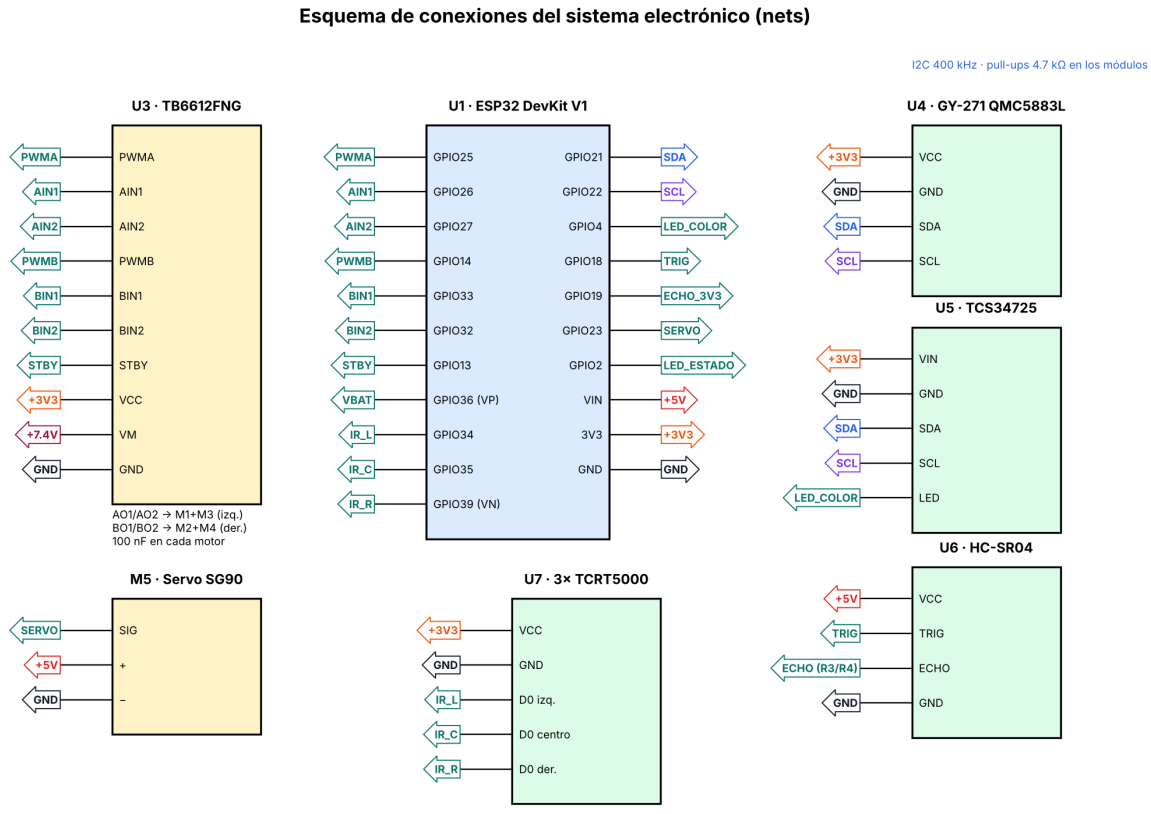


Figura 16. Esquema eléctrico de la etapa de potencia y divisores de tensión.



Nets con el mismo nombre están conectadas. ECHO del HC-SR04 (5 V) pasa por el divisor R3/R4 antes de GPIO19. GPIO34-39 son solo entrada. Se evitan GPIO0/12/15 (pines de arranque).

Figura 17. Esquema de conexiones del ESP32 con los módulos.

Tabla 8. Asignación de pines del ESP32.

Señal	GPIO	Módulo	Nota
PWMA / AIN1 / AIN2	25 / 26 / 27	TB6612 canal A (izq.)	LEDC 20 kHz, 10 bit
PWMB / BIN1 / BIN2	14 / 33 / 32	TB6612 canal B (der.)	GPIO12 evitado (arranque)
STBY	13	TB6612	Habilita el puente
SDA / SCL	21 / 22	QMC5883L + TCS34725	I2C 400 kHz
LED color	4	TCS34725	Iluminación constante
TRIG / ECHO	18 / 19	HC-SR04	ECHO con divisor 1k/2k
SERVO	23	SG90	50 Hz, 14 bit
IR izq. / centro / der.	34 / 35 / 39	TCRT5000	Pines solo entrada
VBAT	36	Divisor 100k/33k	8.4 V → 2.08 V
LED estado	2	LED de la placa	Parpadea en AUTO

6.1 Cálculos de diseño

- **Divisor de batería:** V_{ADC} = 8.4 V · 33k / (100k + 33k) = 2.08 V (dentro del rango del ADC con atenuación 11 dB). Corriente de fuga: 8.4 V / 133 kΩ = 63 µA.
- **Divisor de ECHO:** 5 V · 2k / (1k + 2k) = 3.33 V, compatible con la entrada de 3.3 V del ESP32.
- **PWM de motores:** 20 kHz (inaudible) con 10 bits; duty máximo limitado al 80 % (7.4 V × 0.8 ≈ 5.9 V efectivos) para no exceder los 6 V nominales del motor TT.
- **Desacople:** 470 µF en el bus de motores y en 5 V, más 100 nF en bornes de cada motor para reducir ruido que afecta al magnetómetro y al ADC.
- **Servo:** periodo 20 ms, pulso 500–2400 µs ↔ 0–180°.

7. Construcción de los módulos (firmware ESP32)

El firmware (firmware/robot_esp32/, Arduino-ESP32 core ≥ 3.0) está organizado en un archivo por módulo, con todos los parámetros en config.h:

Tabla 9. Estructura del firmware.

Archivo	Responsabilidad
config.h	Pines, umbrales, ganancias, red Wi-Fi y ESP-NOW
motors.h	TB6612FNG: PWM LEDC, mezcla diferencial (arcade), freno activo
compass.h	QMC5883L: lectura de registros, calibración min/max persistida en NVS, filtro vectorial, declinación
color_sensor.h	TCS34725: integración 50 ms, clasificación por vecino más cercano en cromaticidad + brillo, anti-rebote, calibración en pista
sensors.h	HC-SR04 con mediana de 3, servo, TCRT5000 (con inversión de polaridad), batería
robot_esp32.ino	Lazo de control 20 Hz, máquina de estados, API HTTP, ESP-NOW, eventos
control_remoto.ino	Control propio: joystick + botones → paquete ESP-NOW a 20 Hz

7.1 Brújula digital

El rumbo se calcula como atan2(y, x) del campo corregido (offset hard-iron y escala soft-iron obtenidos girando el robot 8 s) más la declinación magnética local (−7.8°). Para evitar el salto 359°→0° el filtro paso-bajo se aplica sobre el vector unitario y no sobre el ángulo. El error de rumbo se normaliza a [−180°, 180°] y alimenta un control proporcional que mantiene la dirección deseada.

7.2 Detección de colores

Cada lectura RGBC se normaliza (r, g, b suman 1) para independizar el tono del brillo; el brillo ($\log_{10}$ C) separa negro de blanco. El color es el de la referencia más cercana; se requieren 3 lecturas iguales (~150 ms) para aceptar un **cambio de sección**, que se cuenta y se publica como evento a la app y al chatbot. En la sección negra la línea guía es blanca, así que el color detectado invierte la polaridad de los TCRT5000.

7.3 Obstáculos y máquina de estados

En modo autónomo, si el ultrasonido mide menos de 30 cm el robot frena, gira el servo para medir a izquierda y derecha, elige el lado más libre, gira 60° con la brújula, avanza, rodea el obstáculo en arco y, al encontrar la línea, se alinea pivotando hacia el lado de la evasión, lo que garantiza retomar la pista **en el sentido original**. Si se pierde la línea, la brújula mantiene el rumbo y luego se busca en espiral.

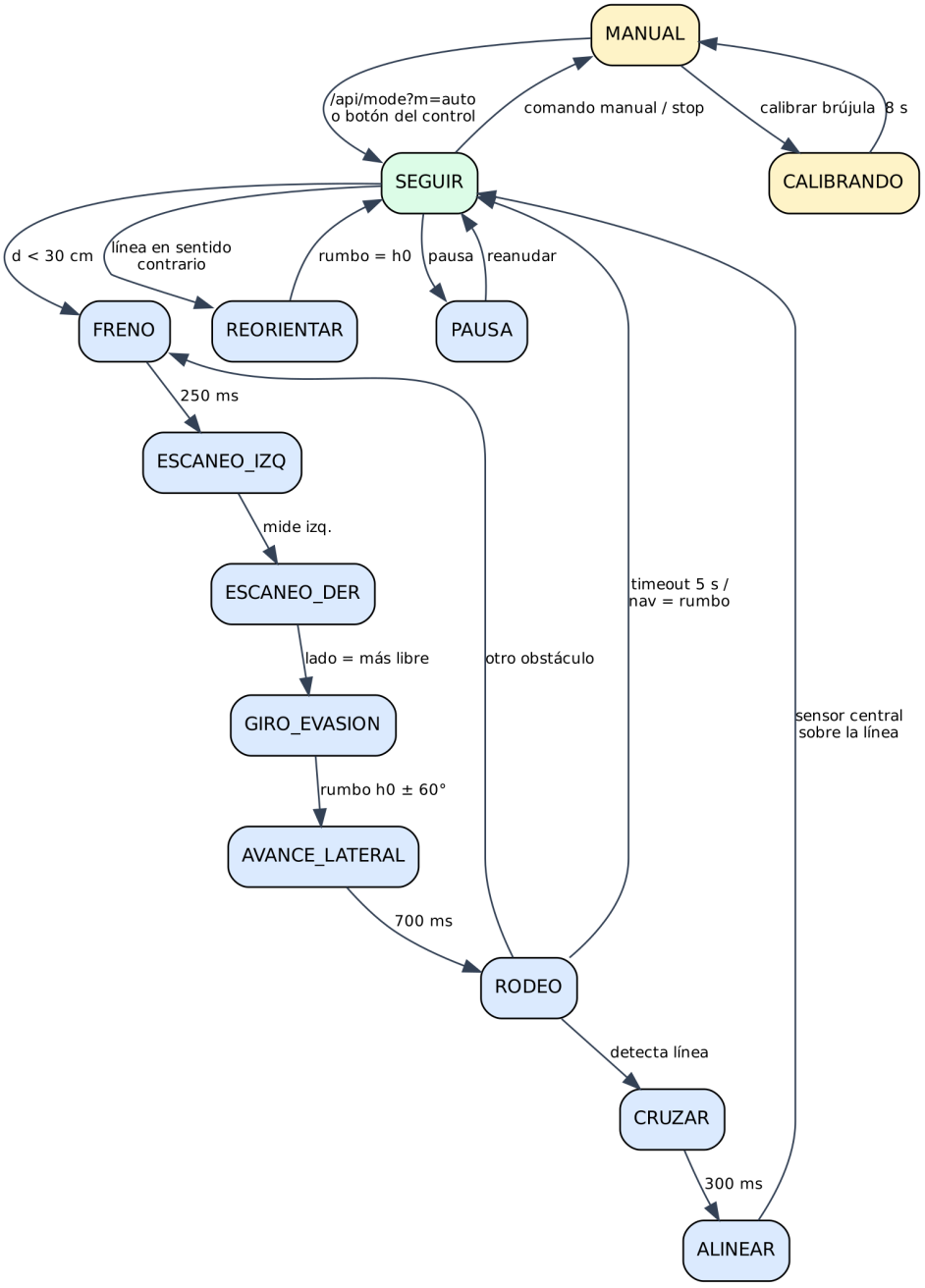


Figura 18. Máquina de estados del firmware.

7.4 Comunicación y conexión con el chatbot

El ESP32 crea la red **ROBOT-ESP32** (SoftAP, 192.168.4.1) y simultáneamente escucha ESP-NOW en el mismo canal. La API REST entrega toda la telemetría en un JSON que la app consulta 2–4 veces por segundo y que se adjunta a cada pregunta hecha a J.A.R.V.I.S.; así el chatbot responde con los valores en tiempo real.

Tabla 10. API HTTP del robot (idéntica en el ESP32 y en el simulador).

Endpoint (GET)	Parámetros	Acción
/api/state	—	Telemetría JSON (rumbo actual y deseado, error, distancia, color, RGBC, línea, PWM, batería, modo, estado, eventos)
/api/cmd	move=forward|backward|left|right|stop, speed=0–100, ms	Movimiento manual por pulsos (se detiene solo al expirar)
/api/mode	m=auto|manual|pausa|reanudar, nav=linea|rumbo	Cambio de modo
/api/heading	deg=0–359 o rel=±grados	Dirección polar deseada
/api/speed	v=20–100	Velocidad del modo autónomo
/api/calibrate	what=compass | what=color&name;=...	Calibración de brújula (giro 8 s) o color

Ejemplo de telemetría: `{"mode":"AUTO","state":"SEGUIR","heading":275.0,"heading_des":274.0,"heading_err":-1.0,"dist_cm":400,"color":"amarillo","rgbc":[1100,1000,420,2100],"line":[0,1,0],"pwm_l":0.55,"pwm_r":0.55,"batt_v":8.19,"obstacles_avoided":2,"event":"Obstaculo superado (#2)"}`

7.5 Seguridad

- Los comandos manuales son pulsos con duración: si la conexión se cae, el robot se detiene.
- Bloqueo de avance si hay un obstáculo a menos de 12 cm en modo manual.
- Botón STOP en la app y en el control remoto (pasa a manual y frena).
- Fusible de 3 A y BMS 2S contra cortocircuito, sobrecarga y sobredescarga.

8. Aplicación Streamlit y chatbot

La app (app/app.py) se conecta por Wi-Fi a la red del robot y usa la API REST. Funciona en el navegador del computador o del celular. Tiene cuatro pestañas:

- **Dashboard:** brújula polar con la dirección actual (azul) y deseada (naranja), métricas de rumbo, error, obstáculo, color y batería; historial de rumbo y distancia; sensores de línea, motores y registro de eventos. Se actualiza con un fragmento de Streamlit cada 0.5 s.
- **Control y voz:** botones Arriba/Abajo/Derecha/Izquierda/Detener, selección de modo y de navegación, envío de la dirección polar deseada, y comandos de voz grabados con el micrófono, reconocidos en español (es-CO) e interpretados por commands.py.
- **J.A.R.V.I.S.:** chatbot que recibe la telemetría actual en cada pregunta. Usa la API de Claude (Anthropic) con respuesta en streaming; sin credenciales o sin internet responde con un motor de reglas local, de modo que la demo nunca queda sin chatbot.
- **Calibración:** calibración de brújula y captura de referencias de color en la pista.

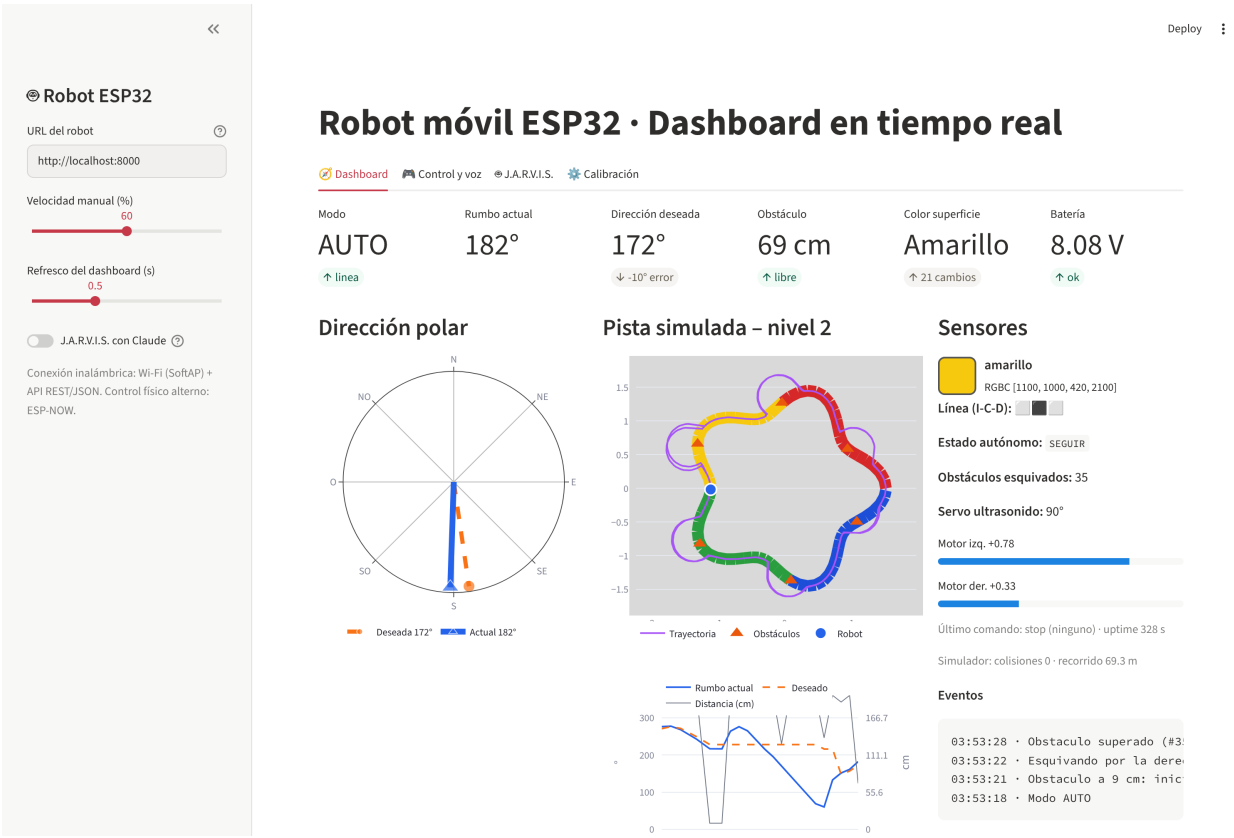


Figura 19. Dashboard en tiempo real conectado al gemelo digital (pista del nivel 2).

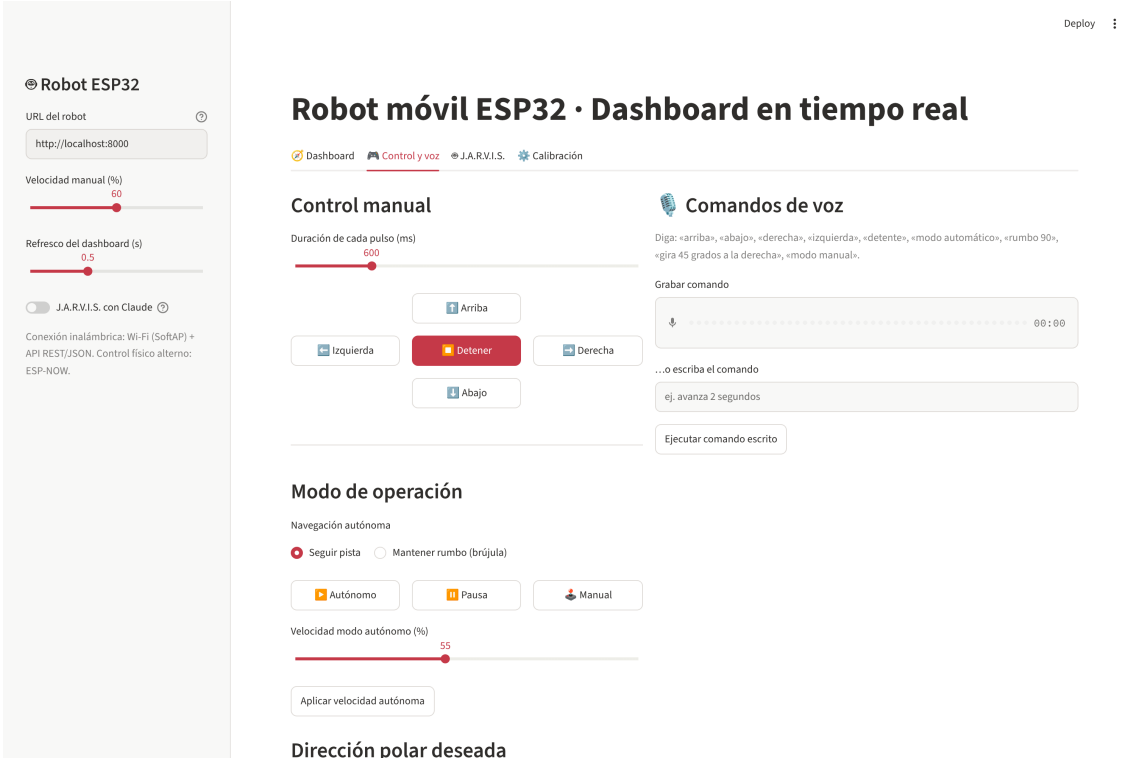


Figura 20. Pestaña de control manual y comandos de voz.



Figura 21. J.A.R.V.I.S. respondiendo con variables en tiempo real.

Tabla 11. Gramática de comandos de voz.

Frase dicha	Acción enviada al ESP32
«arriba», «adelante», «avanza 2 segundos»	/api/cmd?move=forward (600 ms o la duración dicha)
«abajo», «atrás», «retrocede»	/api/cmd?move=backward
«derecha» / «izquierda»	/api/cmd?move=right | left
«detente», «alto», «para»	/api/cmd?move=stop
«modo automático», «modo manual»	/api/mode?m=auto | manual
«rumbo 90», «oriéntate al sur»	/api/heading?deg=90 | 180
«gira 45 grados a la derecha»	/api/heading?rel=45

9. Validación

9.1 Gemelo digital

app/simulator.py implementa la misma API y la misma máquina de estados del firmware con un modelo cinemático de tracción diferencial, brújula con ruido, ultrasonido con haz de ±15°, tres sensores de línea, sensor de color por sección y obstáculos (conos de 5 cm) sobre la línea. Sirvió para ajustar la estrategia de evasión: las primeras versiones perdían la pista o la retomaban en sentido contrario en las curvas de la estrella; la versión final (rodeo en arco + cruce + alineación hacia el lado de la evasión) completa las tres pistas.

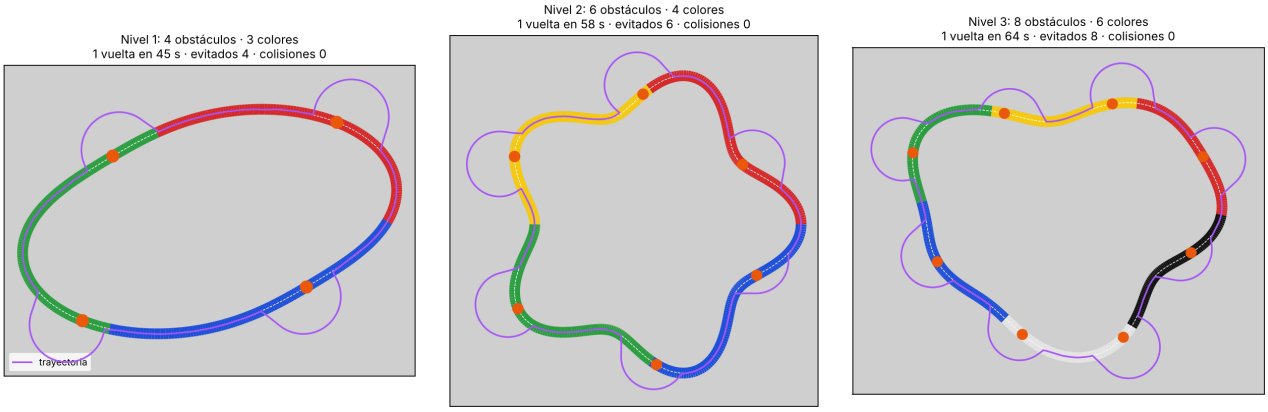


Figura 22. Una vuelta autónoma en cada nivel de la meta final; en morado la trayectoria del robot.

Tabla 12. Resultados en simulación (se reproducen con docs/generar_diagramas.py).

Nivel	Obstáculos	Colores	Tiempo de vuelta	Esquivados	Colisiones	Recorrido
1	4	3	45 s	4	0	10.0 m
2	6	4	58 s	6	0	12.3 m
3	8	6	64 s	8	0	13.0 m

9.2 Pruebas automáticas

app/tests/ contiene 22 pruebas (pytest): 17 del intérprete de voz y 5 del simulador (una vuelta sin colisiones por nivel, parada automática del modo manual y recorrido completo por la API HTTP con el mismo cliente que usa la app). Todas pasan. El firmware se compila en verificación de sintaxis con cabeceras simuladas de Arduino.

9.3 Plan de pruebas físicas (siguiente corte)

Tabla 13. Plan de pruebas sobre el robot físico.

Prueba	Procedimiento	Criterio de aceptación
Brújula	Calibrar; comparar con brújula del celular en 8 rumbos	Error ≤ 5°
Rumbo deseado	Enviar 0°, 90°, 180°, 270° desde la app en modo rumbo	Error estable ≤ 8°
Color	20 lecturas por color en la pista real tras calibrar	≥ 95 % de aciertos
Obstáculos	Cono a 50 cm en recta y en curva, 10 repeticiones	0 colisiones, retoma la pista
Voz	10 comandos por palabra, 2 personas	≥ 90 % reconocidos
Latencia	Cronometrar app → movimiento (video 240 fps)	< 150 ms
Niveles 1–3	Recorrido completo de cada pista	Completa sin intervención

10. Estado y próximos pasos

Tabla 14. Avance del proyecto.

Actividad	Estado
Requerimientos, arquitectura, diagramas de bloques, materiales	Completo
CAD paramétrico, planos, DXF/STL	Completo
Esquemas eléctricos y asignación de pines	Completo
Firmware de todos los módulos + control remoto	Completo (verificado en simulación; falta carga en la placa)
App Streamlit + voz + J.A.R.V.I.S.	Completo (probado con el gemelo digital)
Compra de componentes, corte láser e impresión 3D	Siguiente corte
Ensamble, carga del firmware y calibración en pista	Siguiente corte
Pruebas físicas por nivel (Tabla del plan de pruebas)	Siguiente corte

11. Conclusiones

- Separar los dominios de potencia (7.4 V motores / 5 V–3.3 V lógica) y ubicar el magnetómetro en un mástil no metálico son las dos decisiones de diseño que más impactan la confiabilidad de la brújula y del ESP32.
- Normalizar la cromaticidad del TCS34725 y usar su LED propio permite distinguir los seis colores con un clasificador simple y calibrable en la pista.
- Un gemelo digital con la misma API del firmware permitió desarrollar la app, el chatbot y la lógica autónoma antes de tener el hardware, y detectar fallas (pérdida de la pista y retorno en sentido contrario tras esquivar) que habrían costado mucho tiempo en la pista real.
- La API REST única sirve tanto a la app como al chatbot, de modo que J.A.R.V.I.S. siempre responde con las mismas variables que se ven en el dashboard.

12. Referencias

- Espressif Systems, ESP32 Series Datasheet y documentación de ESP-NOW y LEDC, 2024.
- QST Corporation, QMC5883L 3-Axis Magnetic Sensor Datasheet, rev. 1.0.
- ams-OSRAM, TCS3472 Color Light-to-Digital Converter with IR Filter, datasheet.
- Toshiba, TB6612FNG Driver IC for Dual DC Motor, datasheet.
- Vishay, TCRT5000 Reflective Optical Sensor, datasheet.
- Monolithic Power Systems, MP1584 3A, 1.5MHz, 28V Step-Down Converter, datasheet.
- Streamlit Inc., Streamlit documentation (st.fragment, st.audio_input, st.chat_input).
- Anthropic, Claude API documentation (Messages API, streaming).
- NOAA NCEI, Magnetic Field Calculator (declinación magnética), 2026.
- R. Siegwart, I. Nourbakhsh, D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2.ª ed., MIT Press, 2011.